جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

Data-Driven Offline Optimization of Deep CNN models for EEG and ECoG Decoding

Item Type	Conference Paper
Authors	Tragoudaras, Antonios;Fanaras, Konstantinos;Antoniadis, Charalampos;Massoud, Yehia Mahmoud
Citation	Tragoudaras, A., Fanaras, K., Antoniadis, C., & Massoud, Y. (2023). Data-Driven Offline Optimization of Deep CNN models for EEG and ECoG Decoding. 2023 IEEE International Symposium on Circuits and Systems (ISCAS). https://doi.org/10.1109/iscas46773.2023.10181761
Eprint version	Post-print
DOI	10.1109/iscas46773.2023.10181761
Publisher	Institute of Electrical and Electronics Engineers (IEEE)
Rights	This is an accepted manuscript version of a paper before final publisher editing and formatting. Archived with thanks to IEEE.
Download date	2026-03-05 11:45:12
Link to Item	http://hdl.handle.net/10754/693208

Data-Driven Offline Optimization of Deep CNN models for EEG and ECoG Decoding

Antonios Tragoudaras, Konstantinos Fanaras, Charalampos Antoniadis and Yehia Massoud
Innovative Technologies Laboratories (ITL),
King Abdullah University of Science and Technology (KAUST), Thuwal, Saudi Arabia
{antonios.tragoudaras, charalampos.antoniadis, yehia.massoud}@kaust.edu.sa

***Abstract*—A better understanding of ElectroEncephaloGraphy (EEG) and ElectroCorticoGram (ECoG) signals would get us closer to comprehending brain functionality, creating new avenues for treating brain abnormalities and developing novel Brain-Computer Interface (BCI)-related applications. Deep Convolutional Neural Networks (deep CNNs) have lately been employed with remarkable success to decode EEG/ECoG signals. However, the optimal architectural/training parameter values in these deep CNN architectures have received little attention. In addition, new data-driven optimization methodologies that leverage significant advancements in Machine Learning, such as the Transformer model, have recently been proposed. Because an exhaustive search on all possible architectural/training parameter values of the state-of-the-art deep CNN model (our baseline model) decoding the motor imagery EEG and finger tension ECoG signals comprising the BCI IV 2a and 4 datasets, respectively, would require prohibitively much time, this paper proposes a model-based optimization technique based on the Transformer model for the discovery of the optimal architectural/training parameter values for that model. Our findings indicate that we could pick better values for the architectural/training parameters of the baseline model, enhancing the accuracy of the baseline model by 3.4% in the BCI IV 2a dataset and by 29.8% in the BCI IV 4 dataset.**

***Index Terms*—CNN, EEG Signals, NAS, BCIC IV 2a and 4 datasets**

I. Introduction

The extraction of relevant information from EEG recordings using deep Convolutional Neural Network (CNN) techniques has demonstrated promising results in detecting neurological illnesses such as epilepsy [1] and its treatment feasibility [2], identifying sleep disorders [3], and supporting stroke rehabilitation via Brain-Computer Interface (BCI) [4], [5]. However, despite the tremendous advances in CNN-assisted EEG analysis, optimizing the EEG-decoding-CNN models' parameters has received little attention. Because we cannot derive a closed-form formula for the evaluation of a deep CNN model (we say that we consider it as a " black box"), and on top of that, the evaluation of a deep CNN model is time-consuming because it involves the model training/test on given training/test sets, offline model-based optimization is an attractive method for optimizing the EEG-decoding-CNN models architectural/training parameters.

In offline model-based optimization, a surrogate model of a complex system [1] (like the evaluation of deep CNN consisting of many parameters which in turn take many possible values) is first derived and then used as a substitute for the original system (of the deep CNN model evaluation), usually in an evolutionary algorithm that performs the optimization. As a result, the optimization needs less time, or the evolutionary algorithm can make a more extensive search space exploration, arriving at better solutions. Offline model-based optimization has proven very effective in biology problems [6]–[8]. At the same time, the latest advancement in Machine Learning (ML), such as the Transformer model, has raised the idea of building an offline data-driven surrogate model for optimizing hardware accelerators [9].

In this paper, we exploit a data-driven offline optimization technique based on the Transformer model for building the surrogate model. Using that surrogate model, we optimize the state-of-the-art deep CNN model decoding the motor imagery EEG signals in the BCI IV 2a datasets [10] and the finger tension ECoG signals in the BCI IV 4 dataset [11]. In order to do that, we first perform a random sampling on the baseline model architectural/training parameters space under consideration to create a dataset that matches these random architectural/training parameter values to the evaluated accuracy of the baseline model. Then, we provide that dataset to the proposed model-based optimization methodology to find the architectural/training parameter values that achieve the best accuracy.

The rest of the paper is organized as follows: Section II describes the baseline deep CNN model architecture and the motor imagery EEG and finger tension ECoG signals datasets. Section III explains the proposed data-driven offline optimization methodology to find the best values for the baseline CNN model architectural/training parameters. Finally, in section IV, we demonstrate the better accuracy we can obtain using the architectural/training values found by the proposed optimization methodology, while conclusions are drawn in Section V.

II. Motor Imagery EEG/ECoG signals Decoding

The authors of [12] demonstrated a variety of CNN architectures, ranging from 2- to 31-layer CNN networks, for decoding the EEG signals in the motor imagery BCI IV 2a dataset. Furthermore, these deep CNN models could decode motor imagery EEG signals data as well as or better than Filter Bank Common Spatial Patterns (FBCSP) [13], the winner of many EEG-decoding competitions. Finally, the Shallow ConvNet model was the most accurate on average for all subjects among all the proposed architectures in that work.

[1] A system that requires a long time to respond at a given input and the input space is gigantic.

A. BCI IV 2a dataset

The 22-electrode EEG recordings comprising the BCI IV 2a dataset represent nine patients' motor imagery-related brain activity. Each individual i has a distinct collection of EEG recordings made while performing four distinct body movements, namely *"Left Hand"*, *"Right Hand"*, *"Feet"* and *"Rest"*. The j-th entry of the i-th subject in the dataset is represented as $D^i = \{X^j, Y^j\}$, j from 1 to the total number of recorded trials, where $X^j \in \Re^{E \times T}$ and $Y^j \in \Re^T$, with E denoting the number of recorded electrodes, T the number of timesteps after the discretization of each trial, and Y^j being labeled with one of the four types of body movements, in each timestep of the j-th trial.

B. BCI IV 4 dataset

The shallow ConvNet model has also been tested on Electrophysiological (ECoG) signals in the BCI IV 4 dataset showing better performance than previous works.

The BCI IV 4 dataset consists of 62, 48, and 64 -electrode ECoG recordings corresponding to 3 epileptic patients. The dataset has the same structure as the BCI IV 2a. However, measurements of finger flexion (continuous values) are given at every timestep instead of class labels for every trial.

C. Shallow ConvNet

Shallow ConvNet mimics the spatial filters (bandpass and Common Spatial Patterns (CSP)) in FBCSP with temporal and spatial convolution in the initial two layers. A squaring nonlinearity layer, a mean pooling layer, and a dense layer with the softmax activation function comprise the remainder of the pipeline, which resembles the logarithmic variance calculation in FBCSP. Figure 1 illustrates the Shallow ConvNet model architecture. Although Shallow ConvNet is accurate enough, its accuracy can be improved further by tuning its architectural/training parameters.

III. Data-Driven Optimization of the Shallow ConvNet Architecture

To improve the Shallow ConvNet architecture, we alter some of the Shallow ConvNet architectural/training parameters. First, we assume a vast search space consisting of 8.5 billion distinct architectural/training configurations to exhaust our options for finding a more accurate Shallow ConvNet-like model. Then, using a small sample of Shallow ConvNet-like modified models, $\mathcal{D} = \{\{x_1, y_1\}, .., \{x_i, y_i\}, ..., \{x_n, y_n\}\}$, with x_i being a Shallow ConvNet architecture with random architectural/training parameters values, and y_i being the accuracy of that model after training and testing it on a given training/test set, we train a surrogate model, based on the Transformer model, predicting the accuracy of new Shallow ConvNet architectures. Finally, we employ the Firefly evolutionary optimization algorithm using the previous surrogate model to explore the Shallow ConvNet architectural/training space and discover their best values that maximize the accuracy of the Shallow ConvNet model [12].

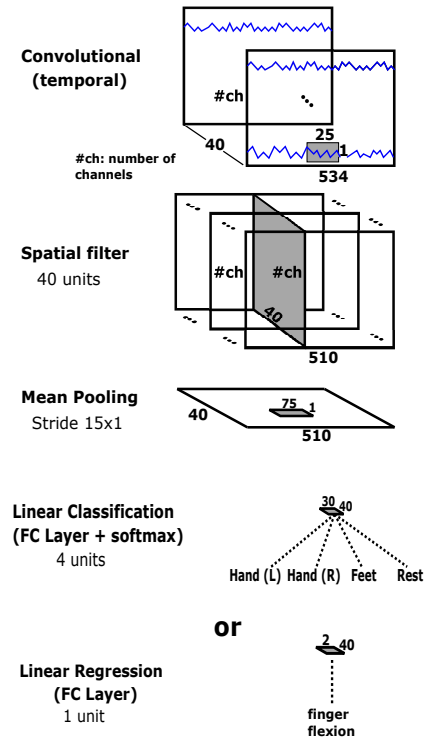

Fig. 1. The Shallow ConvNet model architecture decoding the BCI IV 2a or 4 datasets, respectively.

TABLE I
Architectural/Training parameters of Shallow ConvNet defining our search space.

Architectural/Training parameter p_i	Num. of discrete values	Actual range of values *[Start:End:Step]*
Temporal Filters	30	[25 : 54 : 1]
Temporal Filter Length	26	[10 : 35 : 1]
Spatial Filters	29	[25 : 53 : 1]
Temporal Pool Kernel Length	33	[50 : 82 : 1]
Temporal Pool Stride Length	20	[10 : 29 : 1]
Dropout Probability	7	[0.2 : 0.8 : 0.1]
Learning Rate	9	[1.25e-2 : 9.25e-2 : 1e-2]
Weight Decay	9	[1.125e-4 : 9e-4 : 1.125e-4]

A. Shallow ConvNet Architectural/Training Parameters Search Space

This subsection discusses the architectural/training Shallow ConvNet design choices we considered during our experiments. A ConvNet-like architecture is defined by eight discrete-valued variables (p_i for i from 1 to 8), each corresponding to a different ConvNet architectural/training parameter. As shown in Table I, the first five design choices refer to ConvNet architectural parameters, (see Fig. 1), and the last three are associated with tuning the training of the Shallow ConvNet.

B. Surrogate Model

The purpose of the surrogate model is to predict the accuracy of the Shallow ConvNet model on a given dataset ($\mathcal{D}$) for any combination of the architectural/training parameters shown in Table I. The objective in the training of the surrogate

model is essentially the minimization of the mean-squared-error between the projected $f_\theta(x_i)$ and the ground truth y_i accuracy values. However, such a model can overestimate its predictions at unseen configurations (call them x^- configurations), as shown in [8]. Therefore, for that purpose, we optimize f_θ to discover those x^- configurations and formulate the objective function appropriately to prevent f_θ from learning overestimated values at x^- configurations [14]. Thus, the objective in training is

$$\theta^\star = \underset{\theta}{argmin}\ (\mathbb{E}_{xi,yi\sim D}[(f_\theta(x_i) - y_i)^2] - \alpha\, \mathbb{E}_{x^-\sim Opt(f_\theta)}[f_\theta(x^-)]), \tag{1}$$

where $\mathbb{E}_{xi,yi\sim D}[(f_\theta(x_i) - y_i)^2]$ is the MSE term, $\alpha\, \mathbb{E}_{x^-\sim Opt(f_\theta)}[f_\theta(x^-)]$ is the term in the objective to prevent overestimations of unseen configurations, and Opt is the Optimizer, e.g., the Gradient Ascent (GA), for discovering the x^- configurations.

The following Deep Learning (DL) pipeline (See Figure 2) represents the surrogate model (f_θ):

- The input to the surrogate model is the one-hot encodings of the 8 Shallow ConvNet parameters show in Table I.
- In addition, the one-hot encodings of the Shallow ConvNet architectural/training parameters are embedded into a 64-dim latent space resulting in an 8×64 matrix.
- Then the 8×64 matrix is then given to two multi-Head Attention [15] layers, and the output of the second multi-Head Attention layer is flattened to a 512-dim vector
- After that, the 512-dim vector is fed to N parallel dense layers, spawning $\{f_\theta^1, f_\theta^2, ..., f_\theta^N\}$ outputs.
- Moreover, that 512-dim vector is also given to a 2 dense layers network producing N weights $\{w_1, w_2, \cdots, w_N\}$ that decide the contribution of each f_θ^i.
- The final output is obtained by $f_\theta(x) = \sum_{i=1}^{N} w_i f_{\theta(x)}^i$, where $\sum_{i=1}^{N} w_i = 1$.

To summarize, Algorithm 1 outlines the training of the surrogate model.

Algorithm 1 Surrogate Model Training.

```
Initialize the surrogate model (f_θ0).
for every batch in D do
    Initialize x⁻ values with the x values in the batch
    for _ in range: Opt_iter do
        Update the x_j⁻ values to maximize f_θ(x), ∀j
    end for
    Update θ using the Adam Optimizer with Loss
    function the Eq. 1 and negative samples the x⁻ values
end for
return final learned model f_θ
```

C. Firefly Optimizer

After training the surrogate model with Algorithm 1, we call the Firefly optimizer [16]–[18] to start searching for the architectural/training values that lead to the most accurate Shallow ConvNet architecture. Initially, a population of n different firefly particles, i.e., the Shallow ConvNet Architecture with different parameter values, is selected. Each firefly particle x_i in the population is associated with an intensity value I, i.e., $f_\theta(x_i)$. The intuition behind updating the "position" of a firefly is based on the observation that a firefly with higher intensity I_j located at position j, x_j will attract another firefly located at x_i with a smaller intensity I_i so that x_i will make a move towards x_j. The position update step is performed as follows:

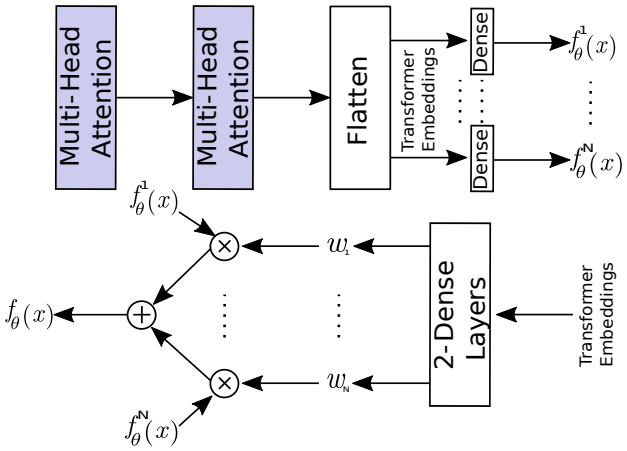

Fig. 2. The DL pipeline of the surrogate model.

$$x_i^{t+1} = x_i^t + e^{-r_{ij}^2}(x_j^t - x_i^t) + \alpha_t \epsilon_i^t, \tag{2}$$

with

$$\alpha_t = 0.97\alpha_{t-1},$$

where r_{ij} is the intensity distance between the firefly particle i and the firefly particle j, ϵ_i^t is a random configuration from the design space, and α_t is a changing parameter in every iteration that controls the diversity of the design solution. Moreover, because all the calculations in Equation 2 involve continuous variables, but the Shallow ConvNet architectural/training parameters we consider discrete variables, we need to map the continuous values to discrete values. Therefore, after performing the calculations in the position update step, we map the updated position x_i^{t+1} to the closest valid discrete value. Then, after repeating the update step for C cycles, the final list of firefly particles is returned as the solution to the optimization problem, which in our case is the Shallow ConvNet architectural/training parameter values that lead to the best accuracy (see Algorithm 2).

IV. Experimental Results

In this section, we demonstrate that the Firefly Optimizer using as objective the surrogate model (explained in subsection III-B) that can predict the accuracy of the Shallow ConvNet model for different sets of its architectural/training parameter values could discover a more accurate Shallow ConvNet model on the BCI IV 2a and 4 datasets.

Our experimental framework was developed in Python using the TensorFlow library. All our experiments were performed on a system with a single NVIDIA V100 with 64GB of RAM and a 3.7GHz Intel Xeon Gold 6154 CPU.

Algorithm 2 Firefly Pseudocode.

Input: Surrogate model $f_\theta(x)$
Initialize the firefly population x_i, $i = 1, 2, \cdots, n$, i.e., random combinations of the Shallow ConvNet model architectural/training values.
$I_i = f_\theta(x_i)$, $i = 1, 2, \cdots, n$
for $t = 1 : C$ **do**
  for $i = 1 : n$ **do**
    for $j = 1 : n$ **do**
      if $I_i^t < I_j^t$ **then**
        Update x_i^t (to x_i^{t+1}) with Eq. 2
        $I_i^{t+1} = f_\theta(x_i^{t+1})$
        if $I_i^{t+1} < I_i^t$ **then**
          $x_i^{t+1} = x_i^t$
          $I_i^{t+1} = I_i^t$
        end if
      end if
    end for
  end for
end for
return $\underset{x_i, \forall i}{argmax}\ f_\theta(x_i)$

A. Training configurations

First, we created a dataset consisting of 8000 ConvNet architectures, each with a different combination of the architectural/training parameter values we consider in this work (by random sampling the Shallow ConvNet architectural/training parameters search space) and their corresponding average accuracy for all patients in the BCI IV 2a and 4 datasets, respectively. We then provided these two datasets to Algorithm 1 to train the surrogate model for 60K gradient steps. On top of that, we considered the following training strategies for the train/validation split during our experiments. Either train on 80% of the worst-performing configurations and validate on the 20% best-performing configurations, or train on 90% of the worst-performing configurations and validate on 10% of best-performing configurations.

B. Discovered models vs ConvNet model

Tables II and III display the highest-accuracy models derived from the proposed method for both high and low-frequency test sets [2] in the BCI IV 2a and the BCI IV 4 datasets, respectively. Also, Table IV depicts the optimum architectural/training parameter values we obtained for the Shallow ConvNet on the two datasets we examined. It can be seen that the proposed data-driven optimization can find a better combination of the ConvNet architectural/training parameter values than the default values, which leads to better accuracy in decoding the EEG/ECoG signals in the BCI challenge datasets. In addition, the proposed optimization ran $\times 7.87 * 10^6$ and $\times 4.7 * 10^6$ days faster than an exhaustive search of all possible architectural/training parameter values for the BCI IV 2a high-frequency and low-frequency sub-datasets, respectively. Similarly, in the BCI IV 4 ECoG dataset, our optimization achieved a speedup of $\times 9.8 * 10^6$ days, which means we can tune the model on different datasets without relying on default architectural/training parameter values. Finally, it is worth mentioning that our approach identified more accurate Shallow ConvNet architectures despite training on design choices associated with decoding accuracies below the baseline accuracy (see the columns "D Best/Mean in Training" [3] of Table II and III).

[2]The EEG signals in the BCI IV 2a dataset are filtered with a 3rd order Butterworth filter, resulting in a low (0-38Hz) and a high (4-38Hz) frequency sub-datasets.

TABLE II
TOP DECODING-TEST ACCURACIES OF DISCOVERED MODELS VS BASELINE SHALLOW CONVNET MODEL ON BCI-IV 2A CHALLENGE.

Freq.	Our Model	Shallow ConvNet	*D* **Best in Training**	*D* **Mean in Training**
Low	77.2%	73.7%	71.7%	69.2%
High	75.7%	71.9%	70.2%	69.1%
Mean Improvement	**×1.0**	**↑3.4%**	**↑5.5%**	**↑7.3%**

TABLE III
TOP DECODING-TEST CORRELATION SCORES OF DISCOVERED MODELS VS BASELINE SHALLOW CONVNET MODEL ON BCI-IV 4 ECOG CHALLENGE.

Freq.	Our Model	Shallow ConvNet	*D* **Best in Training**	*D* **Mean in Training**
1-200 Hz	0.87	0.67	0.73	0.43
Mean Improvement	**×1.0**	**↑29.8%**	**↑19.1%**	**↑102.3%**

TABLE IV
ARCHITECTURAL & TRAINING PARAMETERS OF THE TOP PERFORMING MODELS DISCOVERED ON BCI IV 2A & BCI IV 4 DATASETS RESPECTIVELY.

Arch./Training Params.	BCI IV 2a Low Freq.	BCI IV 2a High Freq.	BCI IV 4
Temporal Filters	25	36	27
Temporal Filter Length	31	30	35
Spatial Filters	49	31	48
Temporal Pool Kernel Filter	62	51	82
Temporal Pool Stride Length	28	29	20
Dropout Prob.	0.8	0.8	0.2
Learning Rate	0.0125	0.0225	0.0125
Weight Decay	23e-4	0.0	68e-4

V. CONCLUSION

This paper determines the ideal architectural/training parameter values for the Shallow ConvNet model, the state-of-the-art deep CNN (our baseline model) for decoding the motor imagery EEG/ECoG signals constituting the BCI IV 2a and 4 datasets. Our findings demonstrate that we can choose better settings for the Shallow ConvNet architectural/training parameters that we consider for optimization in this work, increasing the baseline model's accuracy by 3.4% and 29.8% in BCI IV 2a and 4 datasets, respectively.

[3]"D Best/Mean in Training" columns of Table II correspond to the best/mean accuracy achieved on the training set of dataset D.

REFERENCES

[1] U. R. Acharya, S. V. Sree, G. Swapna, R. J. Martis, and J. S. Suri, "Automated EEG analysis of epilepsy: a review," *Knowledge-Based Systems*, vol. 45, pp. 147–165, 2013.

[2] K. Gadhoumi, J.-M. Lina, F. Mormann, and J. Gotman, "Seizure prediction for therapeutic devices: A review," *Journal of neuroscience methods*, vol. 260, pp. 270–282, 2016.

[3] M. Sharma, J. Tiwari, and U. R. Acharya, "Automatic sleep-stage scoring in healthy and sleep disorder patients using optimal wavelet filter bank technique with EEG signals," *International Journal of Environmental Research and Public Health*, vol. 18, no. 6, p. 3087, 2021.

[4] K. K. Ang, C. Guan, K. S. G. Chua, B. T. Ang, C. Kuah, C. Wang, K. S. Phua, Z. Y. Chin, and H. Zhang, "Clinical study of neurorehabilitation in stroke using EEG-based motor imagery brain-computer interface with robotic feedback," in *2010 Annual International Conference of the IEEE Engineering in Medicine and Biology*. IEEE, 2010, pp. 5549–5552.

[5] A. Ramos-Murguialday, D. Broetz, M. Rea, L. Läer, Ö. Yilmaz, F. L. Brasil, G. Liberati, M. R. Curado, E. Garcia-Cossio, A. Vyziotis *et al.*, "Brain–machine interface in chronic stroke rehabilitation: a controlled study," *Annals of neurology*, vol. 74, no. 1, pp. 100–108, 2013.

[6] J. Fu and S. Levine, "Offline model-based optimization via normalized maximum likelihood estimation," *arXiv preprint arXiv:2102.07970*, 2021.

[7] D. Brookes, H. Park, and J. Listgarten, "Conditioning by adaptive sampling for robust design," in *International conference on machine learning*. PMLR, 2019, pp. 773–782.

[8] B. Trabucco, A. Kumar, X. Geng, and S. Levine, "Conservative objective models for effective offline model-based optimization," in *International Conference on Machine Learning*. PMLR, 2021, pp. 10 358–10 368.

[9] A. Kumar, A. Yazdanbakhsh, M. Hashemi, K. Swersky, and S. Levine, "Data-driven offline optimization for architecting hardware accelerators," *arXiv preprint arXiv:2110.11346*, 2021.

[10] C. Brunner, R. Leeb, G. Müller-Putz, A. Schlögl, and G. Pfurtscheller, "BCI Competition 2008–Graz data set A," *Institute for Knowledge Discovery (Laboratory of Brain-Computer Interfaces), Graz University of Technology*, vol. 16, pp. 1–6, 2008.

[11] G. Schalk, J. Kubanek, K. Miller, N. Anderson, E. Leuthardt, J. Ojemann, D. Limbrick, D. Moran, L. Gerhardt, and J. Wolpaw, "Decoding TwoDimensional Movement Trajectories Using Electrocorticographic Signals in Humans," *J Neural Eng*, vol. 4, pp. 264–275, 2007.

[12] R. T. Schirrmeister, J. T. Springenberg, L. D. J. Fiederer, M. Glasstetter, K. Eggensperger, M. Tangermann, F. Hutter, W. Burgard, and T. Ball, "Deep learning with convolutional neural networks for eeg decoding and visualization," *Human brain mapping*, vol. 38, no. 11, pp. 5391–5420, 2017.

[13] K. K. Ang, Z. Y. Chin, H. Zhang, and C. Guan, "Filter bank common spatial pattern (FBCSP) in brain-computer interface," in *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)*. IEEE, 2008, pp. 2390–2397.

[14] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[16] X.-S. Yang, *Nature-inspired metaheuristic algorithms*. Luniver press, 2010.

[17] X.-S. Yang and S. Deb, "Eagle strategy using lévy walk and firefly algorithms for stochastic optimization," in *Nature inspired cooperative strategies for optimization (NICSO 2010)*. Springer, 2010, pp. 101–111.

[18] X.-S. Yang and A. Slowik, "Firefly algorithm," in *Swarm Intelligence Algorithms*. CRC Press, 2020, pp. 163–174.